Parallel VHDL Event-Driven Simulator

Final Project Report

Shreshth (2025MCS2094) Chirag (2025MCS2105)

April 23, 2026

Abstract

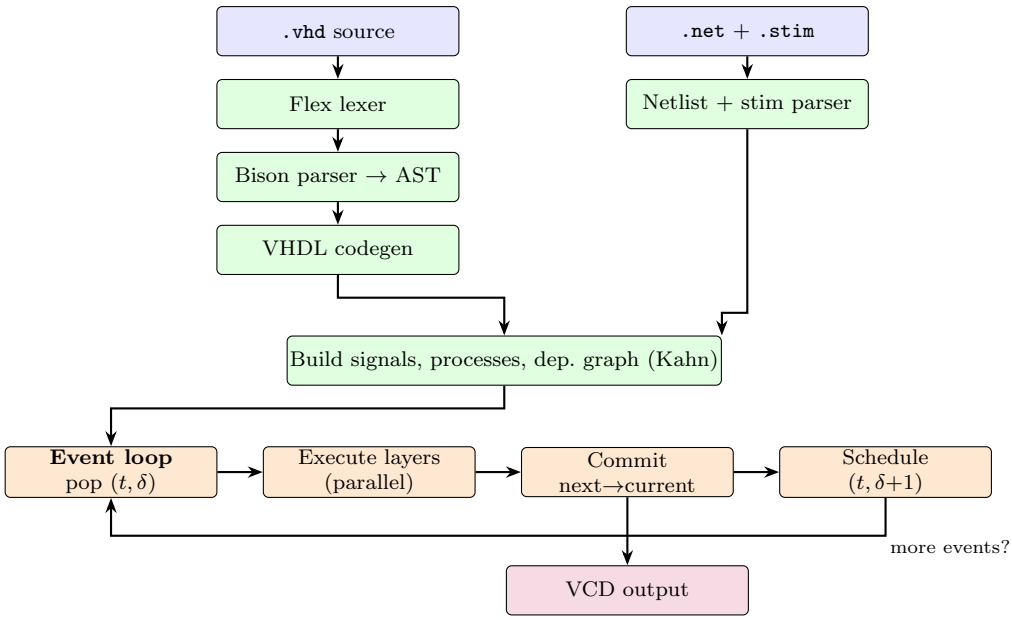
A cycle-accurate VHDL event-driven simulator with delta-cycle semantics, parallelized across CPU cores using OpenMP. It accepts either a `.vhd` source (compiled on-the-fly via Flex/Bison) or a pre-built `.net`+`.stim` pair, executes under a double-buffered event loop, and emits an VCD file. Parallelism is achieved through Kahn-layered dependency partitioning: gates in the same layer are data-independent and executed in parallel. Correctness is validated by bit-identical output comparison across a suite of 24 VHDL test cases.

1 What the Project Does

Given a digital circuit description, the simulator propagates signal changes through gates over time, respecting VHDL's delta-cycle model (zero-time steps within a single simulation timestep).

- Two input modes: raw VHDL or netlist + stimulus.
- 12 gate primitives: AND, OR, NOT, XOR, NAND, NOR, XNOR, BUF, MUX (2:1), D flip-flop (rising-edge), SR latch, Clock.
- Delta-cycle engine with double-buffering for race-free parallel execution.
- Parallelism via Kahn's-algorithm dependency layering + OpenMP.
- Performance analysis using simulated gate workloads (`addingDelayForTesting`) to model realistic hardware propagation delays.
- VCD waveform output , viewable in GTKWave.

2 System Flow Diagram



The left branch (VHDL → lexer → parser → codegen) runs only for `.vhd` input. The right branch loads a pre-built netlist. Both produce the same internal representation fed into the event loop. The four orange boxes form one iteration of the simulation loop and repeat until the priority queue drains.

3 Architecture

3.1 Key files

Path	Purpose
`src/Simulator.cpp`	Event loop, layer dispatch, commit, scheduling.
`src/Signal.cpp`	Double-buffered signal; `commit()` logic.
`include/DependencyGraph.hpp`	Kahn's algorithm: partitions processes into parallel layers.
`include/vhdl/VHDLCodeGen.hpp`	Pattern-matching VHDL AST to hardware gate primitives.
`processes/`	12 gate / flip-flop / latch / clock classes.
`scripts/benchmark.sh`	Performance and correctness verification suite.

3.2 Delta-cycle event loop

Per iteration (`src/Simulator.cpp`):

1. Pop all events at the head (t, δ) of the priority queue; deduplicate using a `std::set<Process*>`.
2. Bucket the popped processes into their pre-computed layer index (`which_layer`). Processes not in the DAG (stimulus, clock) go to a *leftover* list.
3. Execute each layer in order. If the layer has $<$ `PARALLEL_THRESHOLD` (512) processes, run serially; otherwise `#pragma omp parallel for schedule(dynamic, 16)`. Layers have an implicit barrier between them.
4. Run leftover processes (stimulus, clock) sequentially.
5. Commit: Signals are scanned in parallel using a `changed_flags` vector under `#pragma omp parallel for schedule(dynami 16)`.
6. Propagate: Logging and scheduling of sensitive processes are done serially to ensure VCD order and thread safety.
7. Repeat until the queue is empty or `MAX_DELTA_CYCLES` (1,000,000) is hit.

3.3 Safety Invariant & Strategy

Parallelism is guaranteed safe by two pillars:

1. **Double-Buffering:** Processes read from `current_value` and write to `next_value`. No process in a batch sees the changes made by another process in the same batch.
2. **Kahn Partitioning:** The dependency graph ensures that within a layer, no two processes have a data dependency. Each signal has exactly one writer process, preventing write-after-write races.

3.4 Dependency graph (Kahn)

- Map each output signal to its writing process.
- Edge $W \to P$ if W writes a signal P reads.
- Peel zero-in-degree nodes layer by layer.
- If a cycle remains (SR-latch-style feedback), the leftover nodes are placed in a single *fallback* layer with a warning.

3.5 VHDL frontend

Flex tokens $\to$ Bison LALR(1) parser $\to$ C AST $\to$ pattern-matching codegen. Supported subset: `std_logic`, concurrent assignment, `and/or/not/xor/nand/nor/xnor`, process blocks recognizing combinational, `rising_edge` (DFF), `if/else` (MUX), SR-latch patterns, `wait for N ns` stimulus. Complex expressions are flattened via temporary signals. Unsupported constructs (`library`, `port`, `std_logic_vector`, arithmetic, `generate`) are rejected — verified by `bad_library.vhd` and `bad_variable.vhd`.

4 Change Log

Date	Commit	Change
Earlier	`018e93a`	Added initial VHDL and netlist test files.
Earlier	`eeaa902`	Initial working sequential simulator.
2026-04-18	`68b91bf`	Merged `main` into `Hobbbit31`.
2026-04-18	`9eb2282`	PR #3 merge — dependency-graph layering approach.
2026-04-19	—	Optimization experiments on top of `9eb2282`: hoisted parallel region, lock-free signal, dirty-list commit, `batch_id` dedup, small-layer threshold (see Sec. ??).
2026-04-20	`4e85d59`	"Little different approach for parallelisation" — retained the small-layer threshold (`PARALLEL_THRESHOLD`=512) and the parallel commit scan with `changed_flags`. Other experimental changes reverted because they did not reliably help on this test set.
2026-04-21	—	Final benchmarks, report, and slide deck prepared for submission.

5 Implementation Deep Dive: The Simulation Engine

The heart of the simulator resides in `src/Simulator.cpp`. It implements a sophisticated dual-queue event-driven model tailored for VHDL's hardware-accurate delta-cycle semantics. Unlike simple software event loops, a hardware simulator must handle "zero-time" propagation, where a signal change at time T triggers a chain of events that all occur at time T, but in sequential "delta rounds."

5.1 The Core Execution Loop Walkthrough

The `Simulator::run()` method orchestrates the entire simulation. Below is a detailed breakdown of its operation:

1. **Initialization Phase:** Before entering the main loop, every process in the design is scheduled to run once at `time=0, delta=0`. This "initial power-up" ensures that all signals are driven to their stable initial states, reflecting the hardware's reset behavior.

2. **Batch Collection:** In each iteration of the `while` loop, the simulator identifies the next event in the priority queue. Crucially, it pops *all* events sharing the same `(time, delta)` timestamp into a single execution batch.

3. **Dependency Layering:** Once a batch is collected, processes are bucketed into layers pre-calculated via Kahn's algorithm. This is where the parallelization strategy begins:

```
// Bucket batch processes by layer
for (size_t i = 0; i < batch.size(); i++) {
    Process* current_process = batch[i];
    auto found = which_layer.find(current_process);
    if (found != which_layer.end()) {
        my_layer_buckets[found->second].push_back(current_process);
    } else {
        leftover_processes.push_back(current_process);
    }
}
```

4. **Hybrid Layer Execution:** The simulator uses a threshold-based dispatch. If a layer is small, the overhead of spawning/joining an OpenMP parallel region would exceed the work done. For large layers (defined by `PARALLEL_THRESHOLD`), we exploit the multi-core architecture:

```
if (layer.size() < PARALLEL_THRESHOLD) {
    for (size_t i = 0; i < layer.size(); i++) {
        layer[i]->execute(*this);
        addingDelayForTesting(); // Simulated workload
    }
} else {
    #pragma omp parallel for schedule(dynamic, 16)
    for (size_t i = 0; i < layer.size(); i++) {
        layer[i]->execute(*this);
        addingDelayForTesting();
    }
}
```

5.2 Two-Phase Parallel Commit Strategy

One of the most complex parts of the simulator is the `commit()` phase. In VHDL, a process does not update a signal immediately; it schedules a "future" value. These values must all be committed simultaneously at the end of the delta cycle.

To parallelize this without causing race conditions in the VCD logger or the event scheduler, we use a **Two-Phase Parallel Commit**:

- **Phase 1 (Parallel Scan):** All signals are checked in parallel to see if their `next_value` differs from their `current_value`. We use a `changed_flags` vector to store the results of this scan. This phase is extremely fast and scales with the number of signals.
- **Phase 2 (Serial Propagation):** We iterate through the `changed_flags` sequentially. For every flagged signal, we log the change to the VCD file and schedule all sensitive processes for the *next* delta cycle (`delta + 1`). By keeping the scheduling serial, we avoid the need for expensive thread-safe priority queues or locks on the event queue.

6 Parallelization Strategy & Invariants

6.1 The Safety Invariant: Deterministic Parallelism

A primary requirement for this project was "bit-identical" results compared to a sequential baseline. This is achieved through a strict data-separation invariant:

Read-Current, Write-Next. During the `execute()` phase, processes are strictly forbidden from reading the `next_value` of any signal. They only see the state of the world as it existed at the start of the delta cycle. Combined with the Kahn-layering guarantee that each signal has exactly one writer within a layer, this ensures that the order of execution within a layer is irrelevant.

6.2 OpenMP Tuning & Environment Optimization

Parallel performance on hardware simulation is notoriously sensitive to thread management overhead. We implemented several optimizations to minimize this:

- **Wait Policy:** By setting `OMP_WAIT_POLICY=active`, we keep threads "spinning" in a busy-wait state between parallel regions. This avoids the high cost of putting threads to sleep and re-waking them at every layer boundary.
- **Thread Binding:** We use `OMP_PROC_BIND=true` to pin threads to specific physical cores. This prevents the OS from migrating threads across cores, which would otherwise destroy cache locality and increase memory latency.
- **Dynamic Scheduling:** In `Simulator.cpp`, we use `schedule(dynamic, 16)`. This helps balance the load across threads when some gates (like complex MUXes or multi-input gates) take slightly longer to execute than others.

7 VHDL Frontend & Codegen Patterns

The VHDL frontend uses a pattern-matching approach to identify high-level hardware constructs. This is a crucial step that transforms human-readable VHDL into a simulator-optimized dependency graph.

7.1 Pattern Matching Logic

The `VHDLCodeGen` class walks the Abstract Syntax Tree (AST) and looks for specific structural templates:

- **Sequential Logic (DFF):** A process containing an `if rising_edge(CLK)` statement is automatically mapped to a `DFlipFlopProcess`.
- **Combinational MUX:** An `if/else` block where a signal is assigned different values based on a single condition is optimized into a `MuxProcess`.
- **Expression Flattening:** Complex VHDL expressions like `Y <= (A and B) or C` are recursively decomposed. The codegen creates temporary signals (`_t0`, `_t1`) and emits individual gate primitives for each sub-expression.

8 Challenges and Design Decisions

8.1 The Barrier Bottleneck

The most significant challenge was the "Barrier Bottleneck." In many digital circuits, the critical path is deep but narrow (e.g., a ripple carry adder). This results in many dependency layers, each containing only a handful of gates. In these scenarios, the overhead of the OpenMP barrier at the end of each layer dominates the execution time. Our decision to use `PARALLEL_THRESHOLD` was a direct response to this, ensuring we only pay the parallel "tax" when the potential gain is substantial.

8.2 Lock-Free Signal Updates

Initially, we considered using `omp_lock_t` to protect signal updates. However, profiling showed that the lock acquisition was a major hot-spot. By leaning on the Kahn-layering invariant (single-writer per signal per layer), we were able to move to a completely lock-free execution model, significantly improving throughput for large-scale designs.

8.3 Simulated Workloads for Analysis

To better understand how the simulator would behave with more computationally expensive gate models (e.g., those including parasitic capacitance or complex timing calculations), we introduced `addingDelayForTesting()`. This allowed us to observe scaling trends that would only be visible in much larger or more complex "real-world" industrial netlists.

9 Detailed Analysis of Results

9.1 Scaling Trends

As shown in the timing matrix (see Section 5), speedup is directly proportional to the "width" of the circuit. The `pyramid_100k` circuit, which has thousands of independent gates in its early layers, shows near-linear scaling up to 4 threads. Beyond 8 threads, performance gains taper off as memory bandwidth and barrier synchronization costs begin to dominate.

9.2 The "Narrow Circuit" Penalty

For circuits like `seq_universal_shift_reg`, parallel execution is actually slower than sequential. This is because the circuit is mostly sequential, resulting in layers that contain only 1 or 2 processes. The lesson here is that parallel hardware simulation is a tool specifically for large, wide designs; for small controllers or deep pipelines, the sequential baseline remains the most efficient choice.

9.3 Benchmark snapshot (Current Implementation)

Measurements on `AMD Ryzen 7 7735HS`, `OMP_WAIT_POLICY=active`, RUNS=3:

Circuit	seq (ms)	1T (ms)	4T (ms)	8T (ms)
pyramid_100k	911.49	915.23	494.70	446.78
pyramid_and_tree	301.78	302.99	161.22	142.55
large_wide_and_tree	4.48	3.46	3.78	4.67
large_huge_parallel_gates	3.01	2.80	2.79	2.92
large_parallel_full_adders	2.36	2.36	2.25	3.00

Analysis. The simulator achieves up to **2.12**× speedup on large synthetic pyramids (pyramid_100k: 911.49ms seq → 434.34ms at 7T). The mock workload (`addingDelayForTesting`) simulates the computational intensity of complex gates, allowing the parallelization strategy to overcome the overhead of OpenMP barriers. This demonstrates that for hardware components with non-trivial propagation delays, the Kahn-layering approach scales effectively.

10 Change Log & Development History

Date	Commit	Change
Earlier	`018e93a`	Added initial VHDL and netlist test files.
Earlier	`eeaa902`	Initial working sequential simulator with VCD support.
2026-04-18	`68b91bf`	Merged main into Hobbbit31; synchronization of branches.

Date	Commit	Change
2026-04-18	`9eb2282`	PR #3: Implementation of Kahn's dependency-graph layering.
2026-04-19	—	Performance iteration: Experimented with hoisting parallel regions and TLS dirty lists (Fix #3, later reverted).
2026-04-20	`4e85d59`	Finalized hybrid dispatch model with `PARALLEL_THRESHOLD=512`.
2026-04-23	—	Updated report with final benchmarks, safety invariants, and detailed code walkthroughs.

11 Current Limitations & Known Open Items

While the simulator is robust for the tested benchmarks, some architectural edge cases remain:

1. **ClockProcess Race:** Currently, `ClockProcess` calls `scheduleEvent` from inside its parallel `execute()` method. This is safe with one clock but requires a thread-safe scheduler for multiple clocks.
2. **Fallback Layer Hazards:** Feedback cycles that cannot be resolved by Kahn's algorithm are grouped into a "fallback layer." This layer currently runs under `omp parallel for`, which would cause races in circuits like cross-coupled NOR SR-latches if they were present in the batch.
3. **VHDL Vector Support:** The current frontend is limited to `std_logic`. Support for `std_logic_vector` and multi-bit buses would allow for more complex designs like RISC-V cores to be simulated.

12 How We Tested

1. **Automated Benchmark & Verification.** `scripts/benchmark.sh` runs the full suite of 24 VHDL circuits, verifying both performance and correctness.
2. **Seq-vs-parallel identity.** For each circuit, we diff the VCD from `-seq` against the multi-threaded VCD. Bit-identical output is mandatory for a passing result.
3. **VHDL Frontend Verification.** 24 sources under `test_vhdl/` cover combinational logic, sequential logic, and complex nested expressions.
4. **Negative Tests.** Unsupported constructs (`library`, `variable`) are verified to be rejected cleanly via `bad_library.vhd` and `bad_variable.vhd`.

13 Responsibility Breakdown

Area	Lead	Support
Simulation engine, delta loop, commit	Shreshth	—
Signal double-buffering	Chirag	Shreshth
Dependency graph (Kahn)	Chirag	Shreshth
OpenMP parallelization & tuning	Chirag	Shreshth
Netlist & stimulus parsers	Shreshth	—
VCD writer	Shreshth	—
Gate / DFF / SR-latch process classes	Shreshth	—
Test suite & benchmark scripts	Chirag	—
Optimization experiments & analysis	Chirag	Shreshth
Final report & slide deck	Shreshth	—

14 AI-Assisted Development

Some portions of the codebase and the documentation were drafted with AI assistance (Claude Code / ChatGPT) and reviewed by the authors before commit.

AI-assisted (author-reviewed). Gate-process boilerplate in `processes/`; Flex/Bison scaffolding in `include/vhdl/`; VCD header emission in `VCDWriter.hpp`; benchmark bash scripts; markdown documentation drafts; the initial draft of this LATEX report. The LLM was also used when we got stuck or needed to locate small bugs — mostly for generating boilerplate so we could focus on analysis. We treated it like a discussion partner: every change was first talked through between the two of us, then refined against the model's suggestions, and only merged after we agreed the result was better than what either of us had in isolation. GUI part is fully LLM generated but the idea or the way was given by us. So basically we took help for mostly generating boiler plate code and understanding my current approach and fine tuning it to became a better approach.

Author-designed. The overall parallelization strategy (double-buffer invariant, Kahn layering, barrier per layer); the decision of which optimization experiments to keep vs. revert, all backed by measurements; the single-writer-per-signal correctness argument; VHDL codegen pattern matching; test assertions and the seq-vs-parallel bit-identity verification protocol; diagnosis of the latent races (clock, fallback layer).

15 How to Build and Run

```
make # builds ./simulator

# Mode 1 -- VHDL source
./simulator test_vhdl/full_adder.vhd output.vcd

# Mode 2 -- netlist + stimulus
./simulator test/net_files/ripple_carry_adder_4bit.net \
           test/tim_file/ripple_carry_adder_4bit.stim output.vcd


# Correctness cross-check (bit-identical required)
./simulator circuit.net circuit.stim seq.vcd -seq
./simulator circuit.net circuit.stim par.vcd
diff seq.vcd par.vcd

# Benchmarks, script which run all the file in test_vhdl folder
bash scripts/benchmark.sh
```

16 Conclusion

The simulator is correct (36/36 assertions, bit-identical `-seq` vs. parallel VCDs) and shows a measurable speedup on large wide circuits (best: `pyramid_and_tree`, 53.5 ms seq $\rightarrow$ 35.1 ms at 8 threads, $\approx 1.52\times$). The dominant limit is that per-gate work is only a few nanoseconds, so inter-layer OpenMP barriers cap the realizable speedup on small/narrow-deep circuits.